# Part II: Graph Algorithms

## Lecture 6: Depth-First Search and Its Applications

Computer Science and Technology
United International College

# Outline

# What are graphs?

A graph is a pair $G = (V, E)$, where

- $V$ is the collection of vertices, and
- $E$ is the collection of edges.

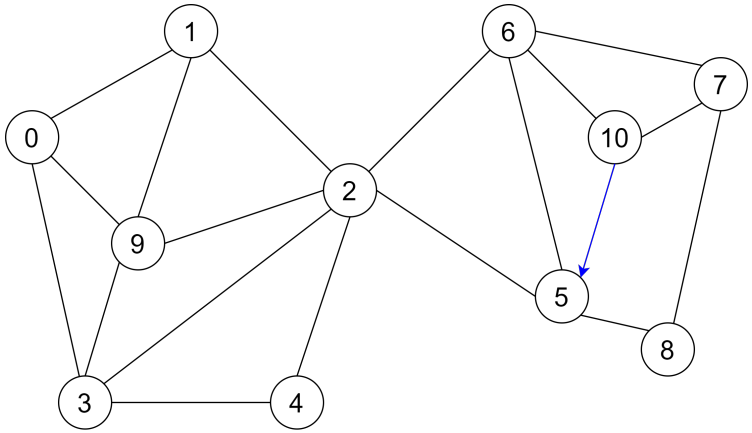
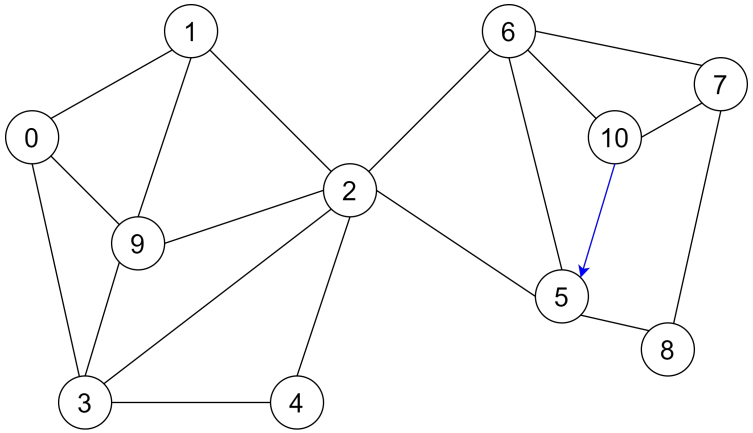



Figure: $V = \{0, 1, 2, ..., 10\}$,
$E = \{(0, 1), (1, 0), (1, 2), (2, 1), ..., (10, 5), ...\}$

- Directed graph:
  $(u, v)$ and $(v, u)$ are distinct for some edges $(u, v) \in E$.
- Undirected graph:
  $(u, v)$ and $(v, u)$ are identical for all edges $(u, v) \in E$.

# Representations of graphs




- Adjacency list representation:
  $adj[u]$ is a linked list of all $v$ such that $(u, v) \in E$.
  - $adj[0] = \{1, 3, 9\}$, $adj[1] = \{0, 9, 2\}$, $\cdots$
  - $5 \in adj[10]$ but $10 \notin adj[5]$ because $(10, 5)$ is directed.
  - In directed graphs, $deg^+(v) = |adj[v]|$ is the out degree of $v$.
- Adjacency-matrix representation:
  $A = [a_{ij}]$ is a $n \times n$ matrix such that $a_{ij} = 1$ if $(v_i, v_j) \in E$.

# Why study graph algorithms

- Graphs are a pervasive data structure in Computer Science.
- Algorithms for working with graph are fundamental to Computer Science.
- Hundreds of interesting computational problems defined on graphs.
- We will sample a few basic ones.

# Objective and Outline

Objective: Discuss an elementary graph algorithm, i.e. depth-first search (DFS).
Reference: Chapter 22 of CLRS

# The DFS Algorithm I

What does DFS do?

- Traverse all vertices in graph, and thereby
- Reveal properties of the graph.

Four arrays are used to keep information gathered during traversal.

1. $color[u]$: the color of each vertex visited
   - white: undiscovered
   - gray: discovered but not finished processing
   - black: finished processing
2. $pred[u]$: the predecessor pointer
   - pointing back to the vertex from which $u$ was discovered
3. $d[u]$: the discovery time
   - a counter indicating when vertex $u$ is discovered
4. $f[u]$: the finishing time
   - a counter indicating when the processing of vertex $u$ (and all its descendants) is finished

# The DFS Algorithm II

How does DFS work?

- It starts from an initial vertex.
- After visiting a vertex, it recursively visits all of its neighbors.
- The strategy is to search "deeper" in the graph whenever possible.

**Algorithm 1** DFS($G$)

```
1:  for all u in V do                          // Initialize
2:      color[u] = white
3:      pred[u] = NULL
4:  end for
5:  time = 0
6:  for all u in V do                          // start a new tree
7:      if color[u] = white then
8:          DFSVisit(u)
9:      end if
10: end for
```

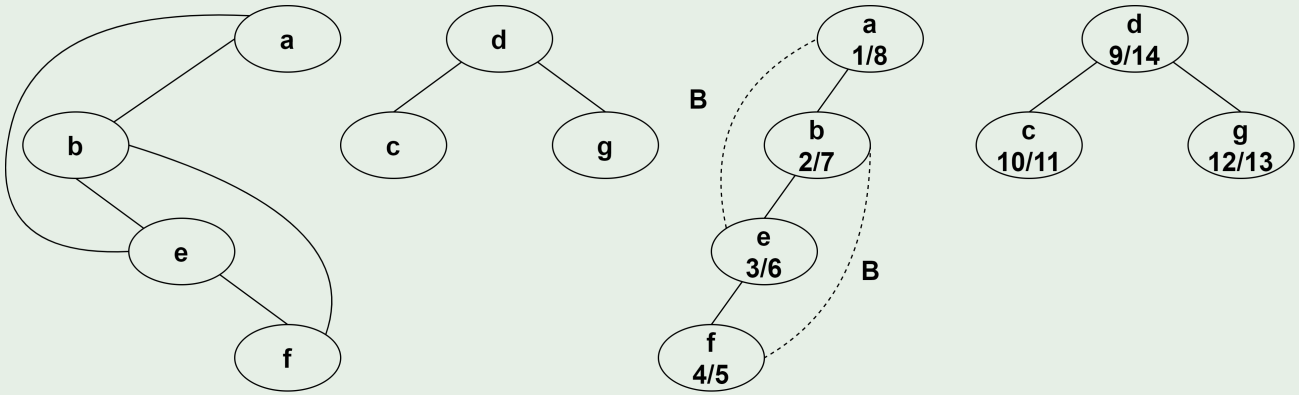
# The DFS Algorithm III

**Algorithm 2** DFSVisit($u$)

```
1:  color[u] = gray                    // u is discovered
2:  d[u] = ++time                      // u's discovery time
3:  for all v in adj(u) do             // Visit undiscovered vertex
4:      if color[v] = white then
5:          pred[v] = u
6:          DFSVisit(v)
7:      end if
8:  end for
9:  color[u] = black                   // u has finished
10: f[u] = ++time                      // u finish time
```

# The DFS Algorithm IV

## Example



# The DFS Algorithm V

The outputs of DFS:

1. The time stamp arrays: $d[v]$, $f[v]$
2. The predecessor array: $pred[v]$

The DFS Forest:

- Use $pred[v]$ to define a graph $F = (V, E_f)$ as follows:

$$E_f = \{(pred[v], v) | v \in V, pred[v] \neq NULL\}$$

- It is a graph with no cycles, and a collection of trees.
- Hence called the The DFS Forest.
- Vertices in the subtree rooted at $u$ are those discovered while $u$ is gray.

# Outline

# Running Time I

**Algorithm 3** $DFS(G)$

```
1:  for all u in V do                    // O(n)
2:      color[u] = white                 //   O(1)
3:      pred[u] = NULL                   //   O(1)
4:  end for
5:  time = 0                             // O(1)
6:  for all u in V do                    // O(n)
7:      if color[u] = white then         //   O(1)
8:          DFSVisit(u)                  //   T_V(G)
9:      end if
10: end for
```

The time cost is

$$T_{DFS}(G) = \mathcal{O}(n) + \mathcal{O}(1) + T_V(G)$$

$T_V(G)$ is the total time cost for all recursive calls of *DFSVisit*. And $n$ is the number vertices in $G$. Each vertex is only visited once. Thus, $T_V(G)$ is not multiplied with $n$

# Running Time II

- For each function call

**Algorithm 4** *DFSVisit*$(u)$

```
1:  color[u] = gray                          // O(1)
2:  d[u] = ++time                            // O(1)
3:  for all v in adj(u) do                   // deg+(u)
4:      if color[v] = white then             //   O(1)
5:          pred[v] = u                      //   O(1)
6:          DFSVisit(v)
7:      end if
8:  end for
9:  color[u] = black                         // O(1)
10: f[u] = ++time                            // O(1)
```

- $deg^+(u)$ is the out degree of the vertex $u$.
- Thus, $T_V(G) = \sum_{u \in V}(2deg^+(u) + \mathcal{O}(1))$.

# Running Time III

Thus, the totoal running time is

$$\begin{aligned} T_{DFS}(G) &= \mathcal{O}(n) + \mathcal{O}(1) + T_V(G) \\ &= \mathcal{O}(n) + \mathcal{O}(1) + \sum_{u \in V} (2deg^+(u) + \mathcal{O}(1)) \\ &= \mathcal{O}(n) + \mathcal{O}(1) + \sum_{u \in V} (2deg^+(u)) + \sum_{u \in V} (\mathcal{O}(1)) \\ &= \mathcal{O}(n) + \mathcal{O}(1) + \sum_{u \in V} (2deg^+(u)) \end{aligned}$$

# Running Time IV

By handshaking theorem,

**Theorem**

*If $G = (V, E)$ is an undirected graph,*

$$2|E| = \sum_{u \in V} (deg(v))$$

*If $G = (V, A)$ is a directed graph,*

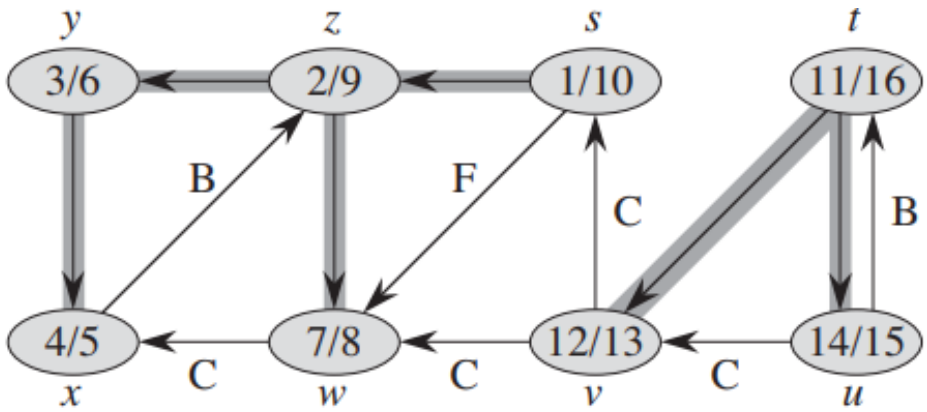
$$|E| = \sum_{u \in V} (deg^-(v)) = \sum_{u \in V} (deg^+(v))$$

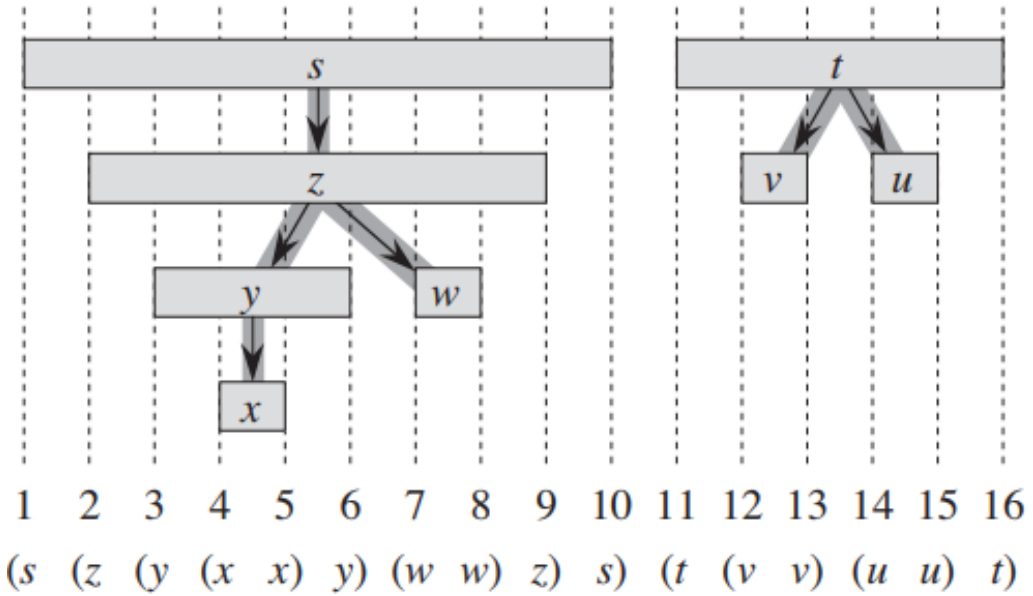
And undirected edge $(u, v)$ in a directed graph is considered as $(u, v)$ and $(v, u)$.
Therefore,

$$\begin{aligned} T_{DFS}(G) &= \mathcal{O}(n) + \mathcal{O}(1) + \mathcal{O}(e) \\ &= \mathcal{O}(n + e) \end{aligned}$$

# Outline

# Time-Stamp Structure I



# Time-Stamp Structure II

- u is a descendant (in DFS trees) of v, if and only if $[d[u], f[u]]$ is a subinterval of $[d[v], f[v]]$.
- u is an ancestor of v, if and only if $[d[u], f[u]]$ contains $[d[v], f[v]]$.
- u is unrelated to v, if and only if $[d[u], f[u]]$ and $[d[v], f[v]]$ are disjoint intervals.

# Time-Stamp Structure III

**Proof.**

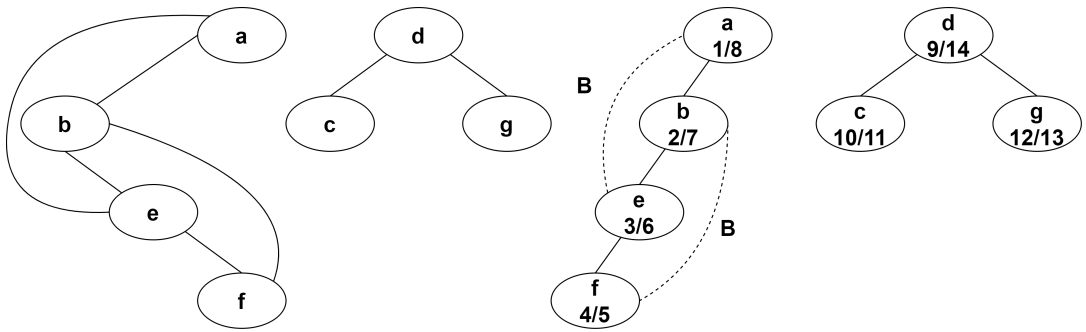
The idea is to consider every case.

Assume $d[v] < d[u]$ without loss of generality.

1. If $f[v] > d[u]$, then
   - $u$ is discovered when $v$ is still not finished yet (marked gray) $\Rightarrow$ $u$ is a descendant of $v$;
   - $u$ is discovered later than $v \Rightarrow u$ should finish before $v$; and
   - hence we have $[d[u], f[u]]$ is a subinterval of $[d[v], f[v]]$.
2. If $f[v] < d[u]$, then
   - obviously $[d[u], f[u]]$ and $[d[v], f[v]]$ are disjoint;
   - which means that when $u$ or $v$ is discovered, the others are not marked gray; and
   - hence neither vertex is a descendant of the other

# Tree Structure I

- Let $G = (V, E)$ be an undirected graph and $F = (V, E_f)$ be the DFS forest of $G$.
- For each $(u, v) \in E$, $(u, v)$ is
  - a tree edge if $(u, v) \in E_f$ or equivalently $u = pred[v]$, i.e. $u$ is the predecessor of $v$ in DFS trees; or
  - a back edge if $v$ is an ancestor (excluding predecessor) of $u$ in the DFS trees.




**Theorem**

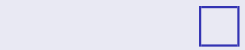
*An edge in an undirected graph is either a tree edge or a back edge.*

# Tree Structure II

Precisely for each edge $(u, v)$ in the undirected graph $G$, $(u, v)$ is either a tree edge or a back edge.

**Proof.**

- Without loss of generality, assume $d[u] < d[v]$.
- Then, $v$ is be discovered while $u$ is gray (why?).
- Hence $v$ is in the DFS subtree rooted at $u$.
  - If $pred[v] = u$, $(u, v)$ is a tree edge.
  - If $pred[v] \neq u$, the $(u, v)$ is a back edge.

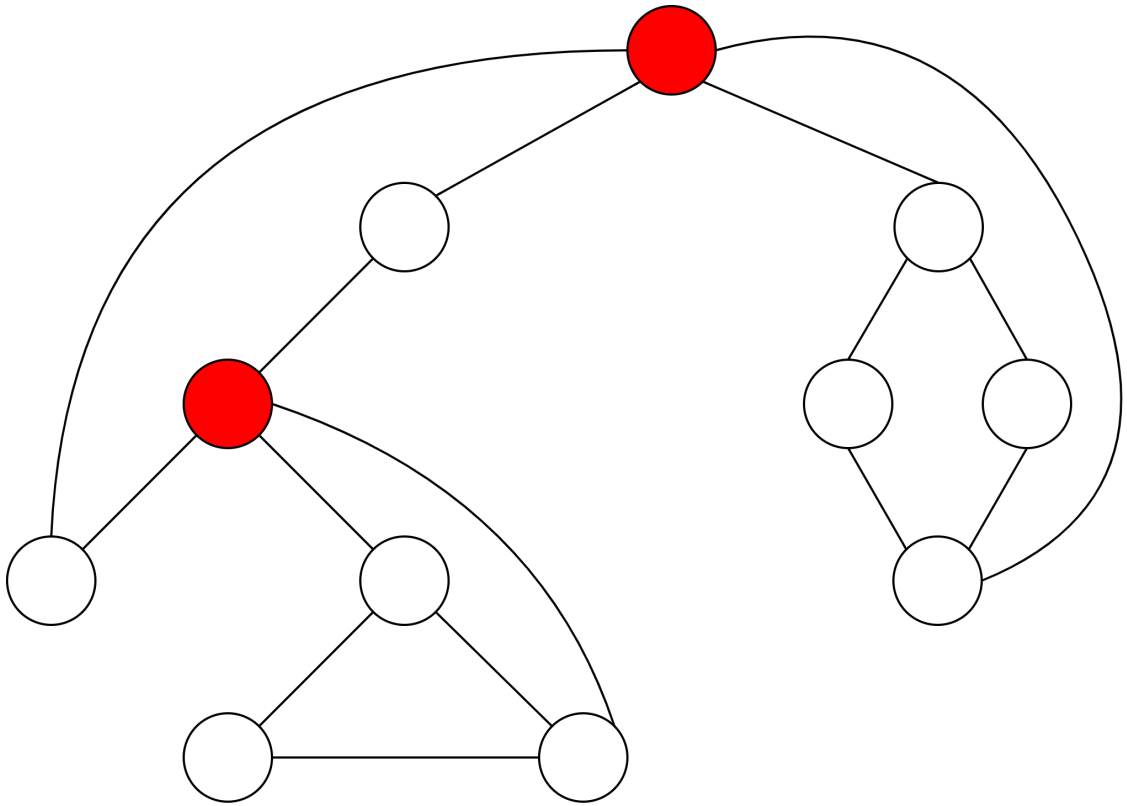


# Outline

# An Application of DFS : Articulation points

## Definition

Let $G = (V, E)$ be a connected undirected graph. An articulation point (or cut vertex) of $G$ is a vertex whose removal disconnects $G$



# A Brute Force Solution

**Question**

Given a connected graph $G$, how to find all articulation points?

A brute-force method:

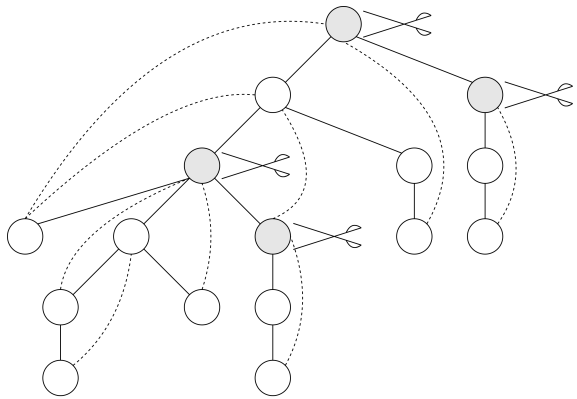
1. Remove a vertex (and its corresponding edges) one by one from $G$.
2. Test whether the resulting graph is still connected or not (say by DFS).
   - DFS on a connected graph produces ONE tree. It's connected.

The running time is $\mathcal{O}(n * (n + e))$.

Can do better by running DFS on $G$.

# Three Observations

1. The root of the DFS tree is an articulation point if it has two or more children.
2. A leaf of the DFS tree is not an articulation point.
3. And for other internal vertex $v$ in the DFS tree, $v$ is an articulation point if
   - if there is one subtree rooted at a child of $v$ that does not have a back edge which climbs "higher" than $v$.



Observations 1 and 2 can be handled easily.

**Question**

How to make use of observation 3?

# Tackle Observation 3

We make use of the discovery time in the DFS tree to define "low" and "high".

- Observe that if we follow a path from an ancestor (high) to a descendant (low), the discovery time is in increasing order.

If there is a subtree rooted at a children of $v$ which does not have a back edge connecting to a vertex with discovery time smaller than $d[v]$, then $v$ is an articulation point.

**Question**

How do we know a subtree has a back edge climbing to an upper part of the tree?

Ans:

- We make use of recursion.

# Tackle Observation 3

- Let $Low(w)$ be the smallest value that a descendant of $w$ can climb up by a back edge.
- We use $d[v]$ to express the "hight" of $v$.
- $v$ is "higher" than $u$ if $v$ and $u$ are on the same branch of the DFS tree and $d[v] < d[u]$.
- If $v$ and $u$ are on different branches, $d[v]$ and $d[u]$ are not compatible by the time-stamp structure.
- Then formally, Observation 3 is

**Observation**

*Let $v$ be an internal vertex and $w_1, w_2, \cdots$ be the children of $v$ in the DFS tree. $v$ is an articulation point if and only if there exist a $w_i$ such that $Low(w_i) \geq d[v]$ for some $i$.*

**Algorithm 5** ArtPt(v)

**Output:** articulation points in the DFS subtree rooted at $v$

```
 1: color[v] = gray
 2: Low[v] = d[v] = ++time
 3: for all w in adj[v] do
 4:     if color[w] = white then
 5:         pred[w] = v
 6:         ArtPt(w)
 7:         if pred[v] = NULL then
 8:             if w is v's second child then
 9:                 output v
10:             end if
11:         else if Low[w] ≥ d[v] then
12:             output v
13:         end if
14:         Low[v] = min(Low[v], Low[w])
15:     else if w! = pred[v] then
16:         Low[v] = min(Low[v], d[w])
17:     end if
18: end for
19: color[v] = black
```

Note:

- The finish time isb not recorded because they are irrelevant for finding articulation points.

# Pseudo-code: Initialization

**Algorithm 5** ArtPt(v)

**Output:** articulation points in the DFS subtree rooted at $v$

```
1:  color[v] = gray
2:  Low[v] = d[v] = ++time
3:  for all w in adj[v] do
4:      if color[w] = white then
5:          pred[w] = v
6:          ArtPt(w)
7:          if pred[v] = NULL then
8:              if w is v's second child then
9:                  output v
10:             end if
11:         else if Low[w] ≥ d[v] then
12:             output v
13:         end if
14:         Low[v] = min(Low[v], Low[w])
15:     else if w! = pred[v] then
16:         Low[v] = min(Low[v], d[w])
17:     end if
18: end for
19: color[v] = black
```

Step2:

- $v$ initially can only climb up to itself.

**Algorithm 5** ArtPt(v)

**Output:** articulation points in the DFS subtree rooted at $v$

```
1:  color[v] = gray
2:  Low[v] = d[v] = ++time
3:  for all w in adj[v] do
4:      if color[w] = white then
5:          pred[w] = v
6:          ArtPt(w)
7:          if pred[v] = NULL then
8:              if w is v's second child then
9:                  output v
10:             end if
11:         else if Low[w] ≥ d[v] then
12:             output v
13:         end if
14:         Low[v] = min(Low[v], Low[w])
15:     else if w! = pred[v] then
16:         Low[v] = min(Low[v], d[w])
17:     end if
18: end for
19: color[v] = black
```

Step6:

- Recursive calls to find all articulation points in each subtree rooted at $w$.

**Algorithm 5** ArtPt(v)

**Output:** articulation points in the DFS subtree rooted at $v$

```
 1: color[v] = gray
 2: Low[v] = d[v] = ++time
 3: for all w in adj[v] do
 4:     if color[w] = white then
 5:         pred[w] = v
 6:         ArtPt(w)
 7:         if pred[v] = NULL then
 8:             if w is v's second child then
 9:                 output v
10:             end if
11:         else if Low[w] ≥ d[v] then
12:             output v
13:         end if
14:         Low[v] = min(Low[v], Low[w])
15:     else if w! = pred[v] then
16:         Low[v] = min(Low[v], d[w])
17:     end if
18: end for
19: color[v] = black
```

Step 7:

- If $v$ is the root, apply Observation 1.

Step 8 and 9:

- If the root has multiple children, it is an articulation point.

# Pseudo-code: Internal Vertices

**Algorithm 5** ArtPt(v)

**Output:** articulation points in the DFS subtree rooted at $v$

```
 1: color[v] = gray
 2: Low[v] = d[v] = ++time
 3: for all w in adj[v] do
 4:     if color[w] = white then
 5:         pred[w] = v
 6:         ArtPt(w)
 7:         if pred[v] = NULL then
 8:             if w is v's second child then
 9:                 output v
10:             end if
11:         else if Low[w] ≥ d[v] then
12:             output v
13:         end if
14:         Low[v] = min(Low[v], Low[w])
15:     else if w! = pred[v] then
16:         Low[v] = min(Low[v], d[w])
17:     end if
18: end for
19: color[v] = black
```

Step 11 - 13:

- If $pred[v] \neq NULL$, $v$ is not a root.
- If $Low[w] \geq d[v]$,
- $v$ has a child $w$ such that all descendants of $w$ cannot climb higher than $v$.
- $v$ is an articulation point by Observation 3.

# Pseudo-code: Update $Low[v]$ from Subtrees

**Algorithm 5** ArtPt(v)

**Output:** articulation points in the DFS subtree rooted at $v$

```
 1: color[v] = gray
 2: Low[v] = d[v] = ++time
 3: for all w in adj[v] do
 4:     if color[w] = white then
 5:         pred[w] = v
 6:         ArtPt(w)
 7:         if pred[v] = NULL then
 8:             if w is v's second child then
 9:                 output v
10:             end if
11:         else if Low[w] ≥ d[v] then
12:             output v
13:         end if
14:         Low[v] = min(Low[v], Low[w])
15:     else if w! = pred[v] then
16:         Low[v] = min(Low[v], d[w])
17:     end if
18: end for
19: color[v] = black
```

Step 14:

- After the recursive call in Step 6, we know the highest position that $w$ or its descendants can climb up to.
- If $w$ or its descendants can climb higher than $Low[v]$, then update.

# Pseudo-code: Update $Low[v]$ from Back Edges

**Algorithm 5** ArtPt(v)

**Output:** articulation points in the DFS subtree rooted at $v$

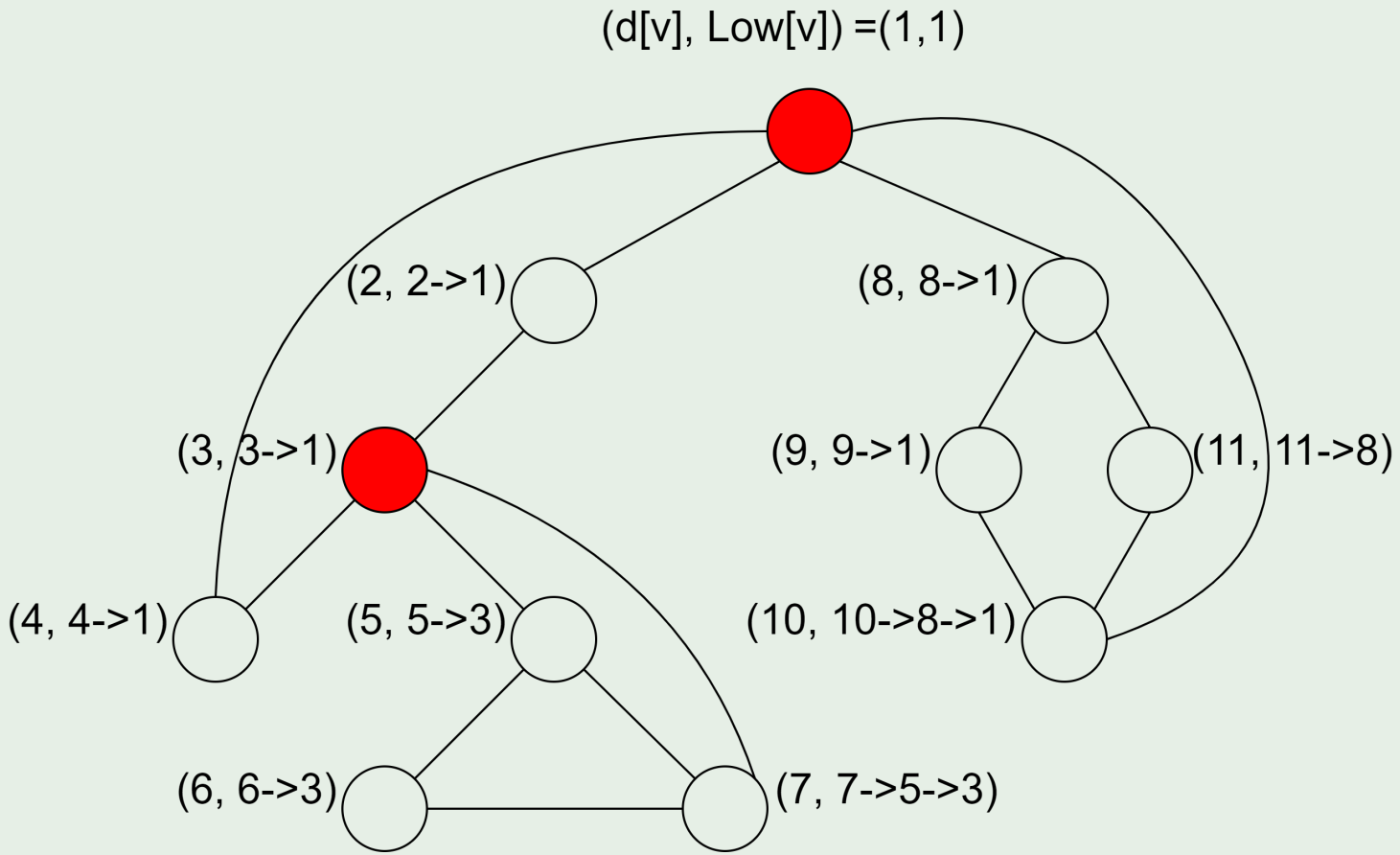
```
 1: color[v] = gray
 2: Low[v] = d[v] = ++time
 3: for all w in adj[v] do
 4:     if color[w] = white then
 5:         pred[w] = v
 6:         ArtPt(w)
 7:         if pred[v] = NULL then
 8:             if w is v's second child then
 9:                 output v
10:             end if
11:         else if Low[w] ≥ d[v] then
12:             output v
13:         end if
14:         Low[v] = min(Low[v], Low[w])
15:     else if w! = pred[v] then
16:         Low[v] = min(Low[v], d[w])
17:     end if
18: end for
19: color[v] = black
```

Step 15 - 17:

- Step 15 is tested if Step 6 fails.
- In other words, $color[w] \neq white$, and $w$ is already discovered.
- Furthermore if $w \neq pred[v]$, $w$ is an ancestor but not a predecessor of $v$.
- Therefore, $(v, w)$ is a back edge.
- $Low[v]$ needs an update if $(v, w)$ climbs higher.

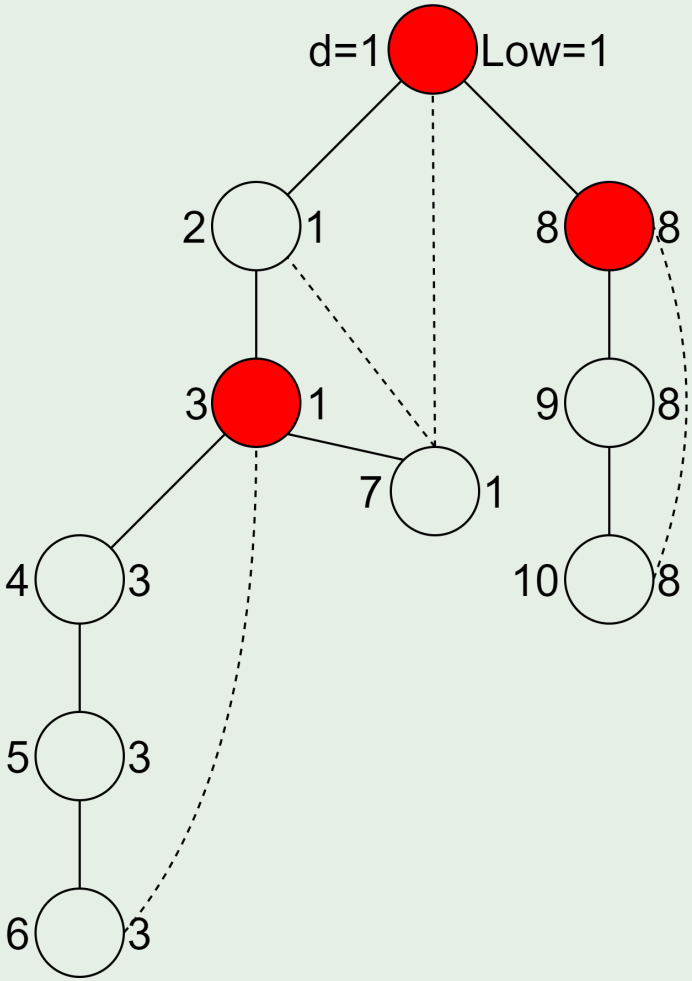
# Articulation Points: Example I

## Example



# Articulation Points: Example II

## Example



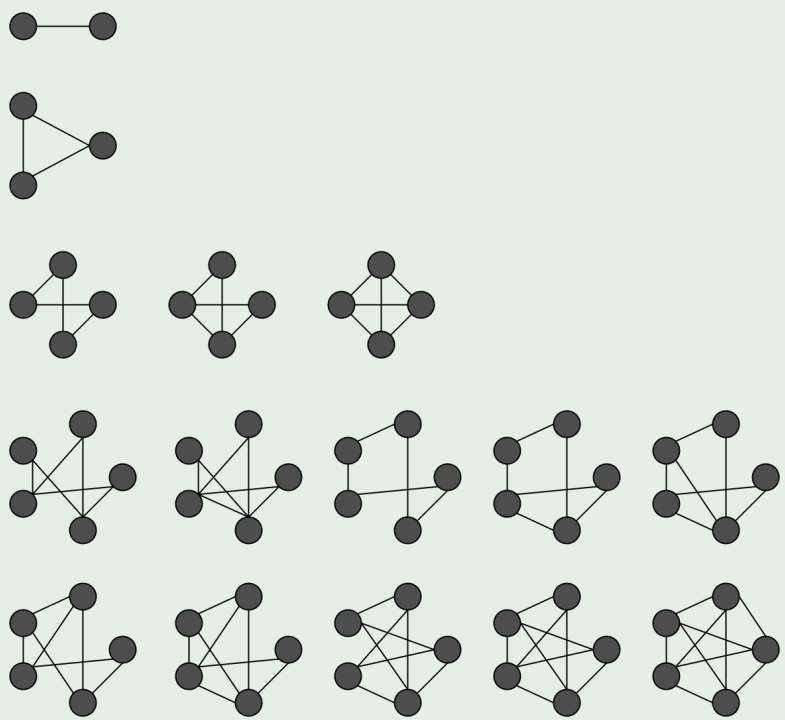
# Outline

# Biconnected Graph I

## Definition

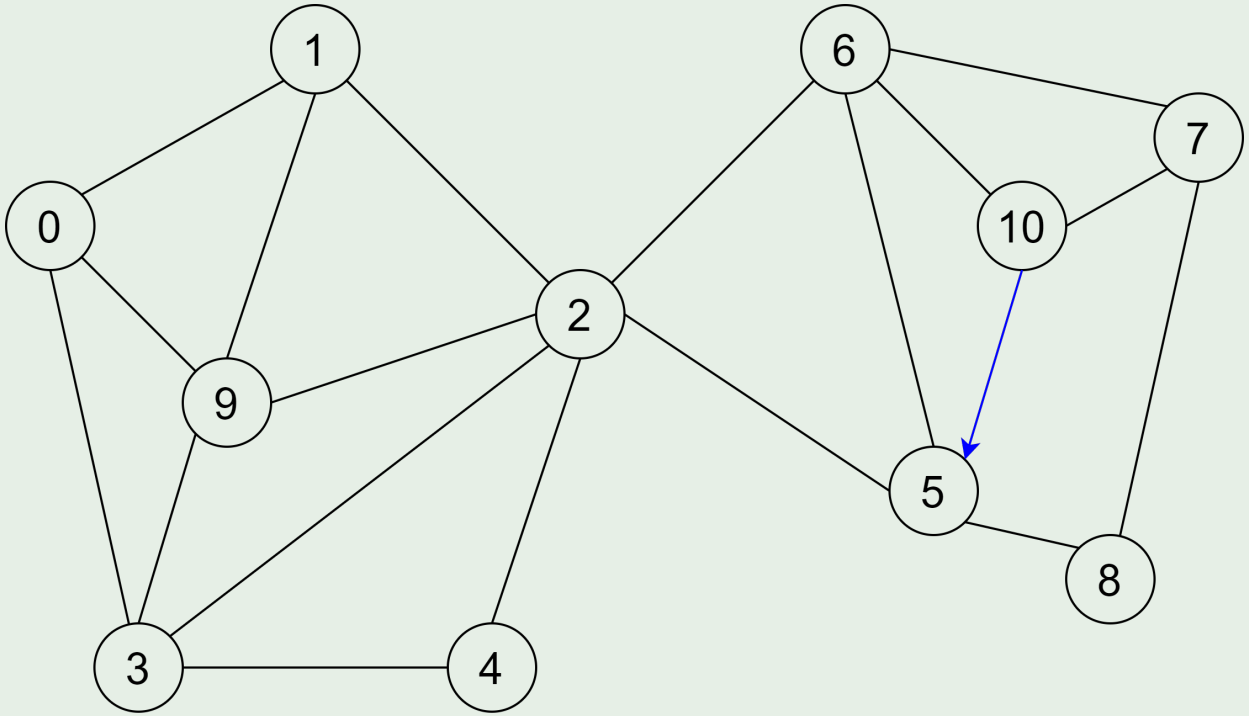
A biconnected graph $G = (V, E)$ is a connected graph which has no articulation point.

## Example (Biconnected graphs?)



# Biconnected Graph II

## Example (Biconnected graphs?)

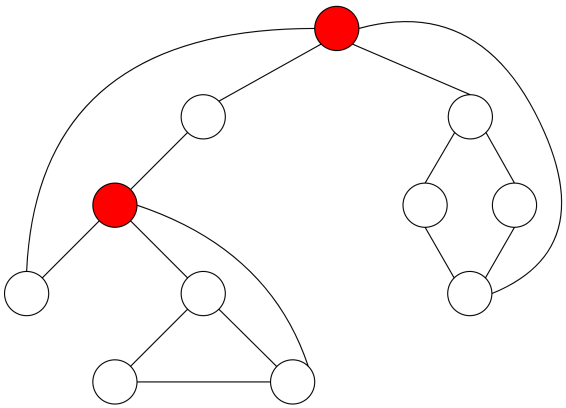


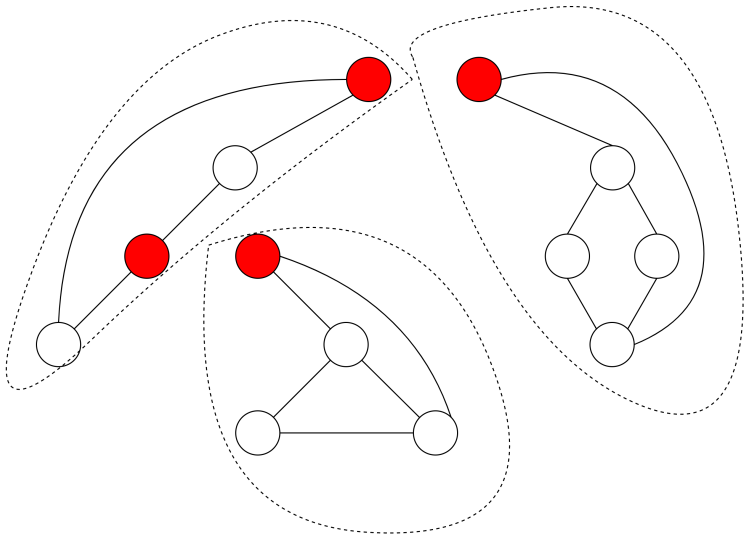

Remark: To disconnect a biconnected graph, we must remove at least two vertices

# An Application of DFS : Biconnected Components I

**Definition**

A biconnected component of a graph $G$ is a maximal biconnected subgraph (i.e., it is not contained in any larger biconnected subgraph) of $G$.





# An Application of DFS : Biconnected Components II

**Question**

How to identify all biconnected components of $G$?

Find all the articulation points of $G$ and check how they separate the biconnected components.

Observations:

- ArtPt enters a component via a tree edge $(u, x)$.
- Once entering a component, it examines all edges in the component before leaving.
- It leaves a component when the vertex $u$ is identified as an articulation point.

# An Application of DFS : Biconnected Components III

Algorithm:

- When we process an edge $(u, x)$, push that edge to a stack.
- Later, if we identify $u$ as an articulation point (where the subtree rooted at $x$ can't climb higher than $u$, then
  - So we pop edges out of the stack until $(u, x)$ (also pop $(u, x)$), those edges belong to a biconnected component.

# Example

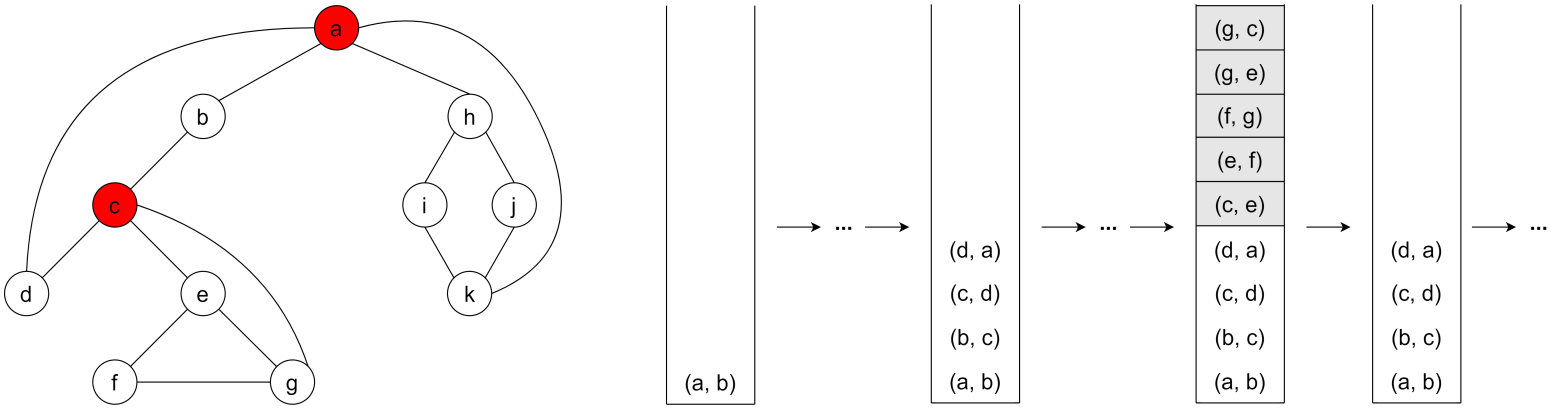



Figure: $c$ is idenfied as an articulation point, as $e$ can't climb higher; pop the stack until $(c, e)$, those edges are in the same biconnected component